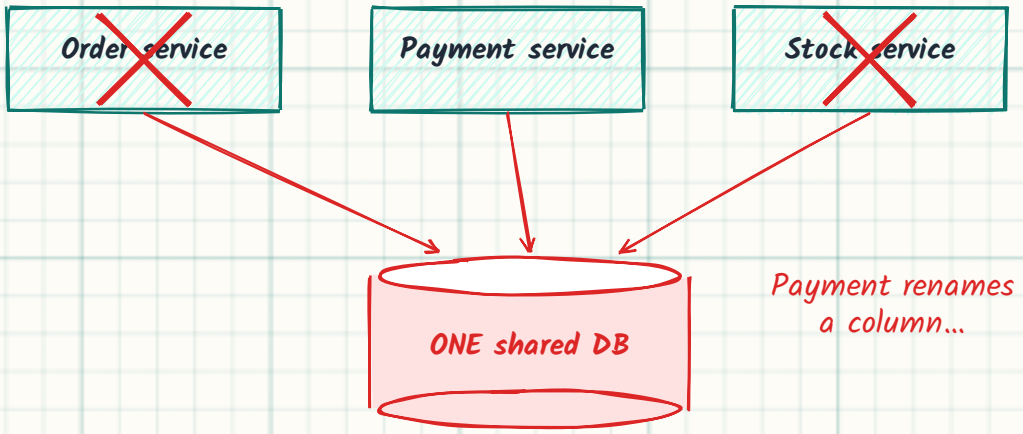
Database per Service

Lesson 10 · data ownership in microservices

In this lesson → ① Shared DB problem ② Database per service ③ 3 privacy levels ④ Golden rule ⑤ Gains vs losses ⑥ Joining across services ⑦ Local copies ⑧ Migrating ⑨ Interview Q&A

Running project: ShopNest, our e-commerce app (users, products, orders, payments, notifications).

① The problem: a shared database

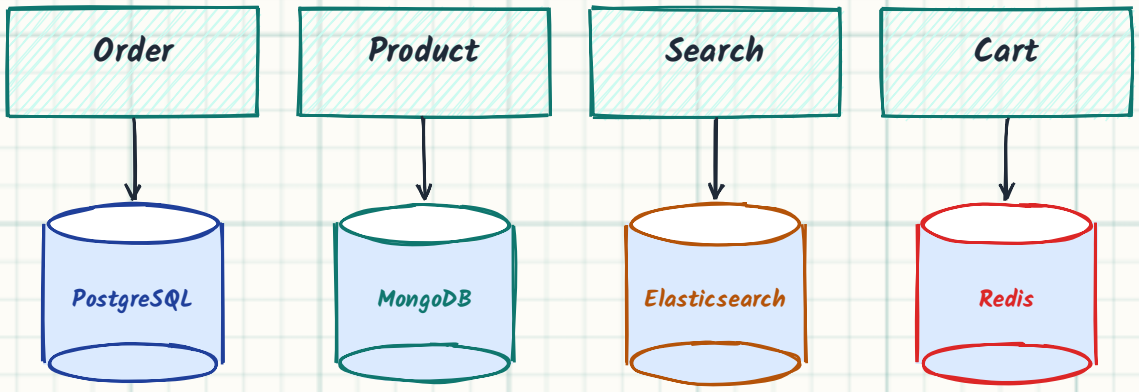


Order & Stock break · deploy together · lock contention

- ✗ Schema change → **other services break**
- ✗ Teams must **release together**
- ✗ Heavy queries **lock** tables others need
- ✗ Split code, shared data = **distributed monolith**

② Database per service

> **Database per service** = each service owns its data privately; other services reach it only through the owner's API or events.

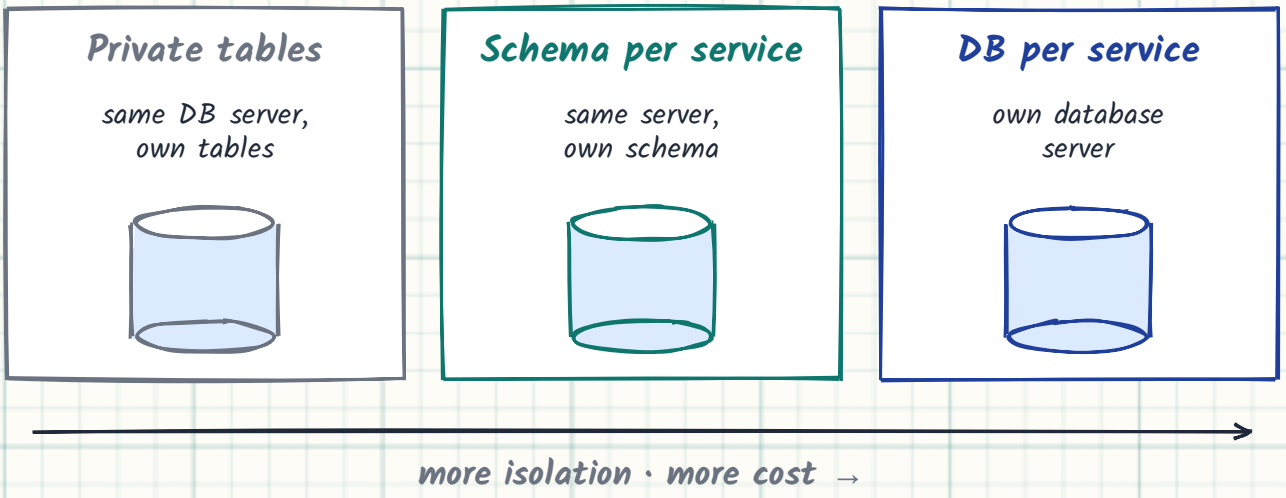


each service owns its data · picks the right store

Service	Store	Why
Order	PostgreSQL	transactions, relations
Product catalog	MongoDB	flexible documents
Search	Elasticsearch	full-text search
Cart	Redis	fast key-value, expiry

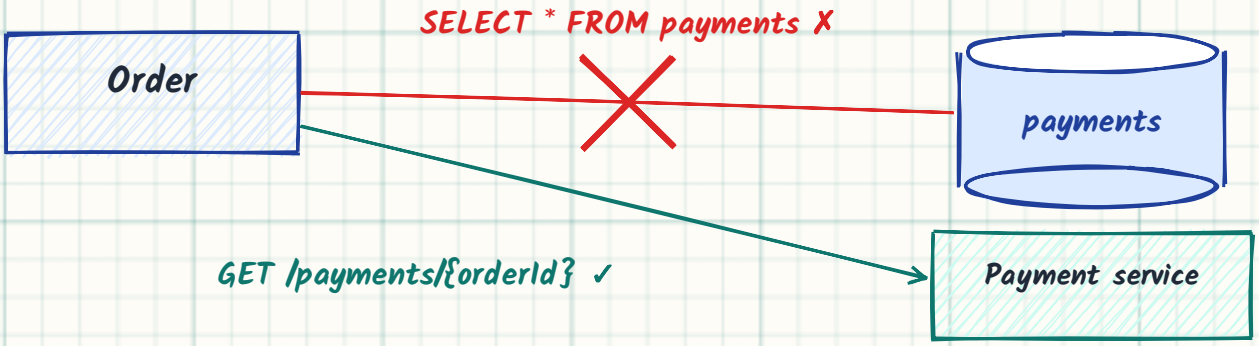
Polyglot persistence
Each service picks the storage that fits its data. Only possible because the data is private.

③ 3 ways to keep data private



Level	Isolation	Cost	When
Private tables	weak (discipline)	lowest	early days, small team
Schema per service	medium (own DB user)	low	good default start
DB server per service	strong	highest	different scale / tech needs

4 The golden rule

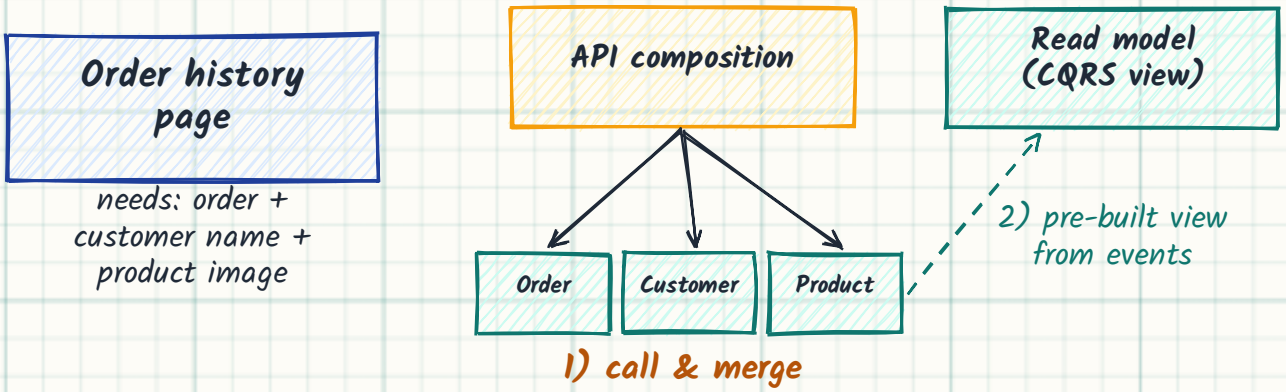


> **Rule**
> Never read or write another service's tables. Use its **API** or its **events**.

5 Gains vs losses

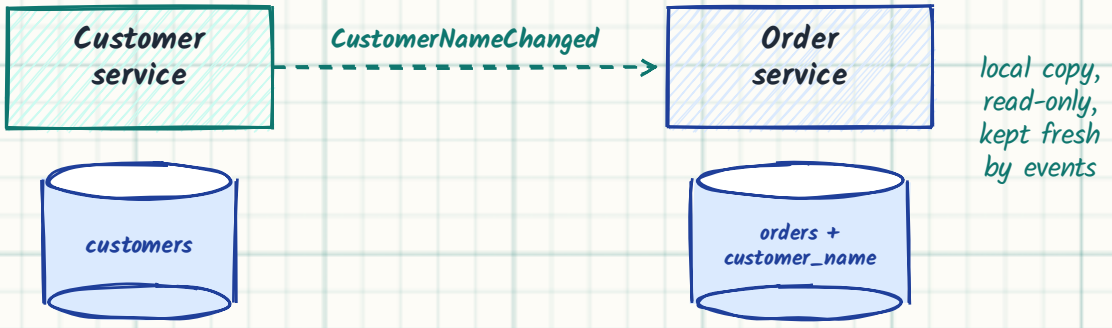
- ✓ Loose coupling
- ✓ Independent scaling
- ✓ Right tool per service
- ✓ Fault isolation
- ✗ No cross-service JOINs
- ✗ No single ACID transaction (→ Saga)
- ✗ Duplicated data
- ✗ Eventual consistency

6 How to "join" across services



Technique	How	Trade-off
API composition	caller fetches from each service and merges	simple; slower, fails if one service fails
Read model (CQRS)	listen to events, build a query-ready view	fast reads; eventual consistency (Lesson 12)
Local copy	keep a read-only copy of a few fields, updated by events	fast; owner stays the source of truth

7 Local copies via events



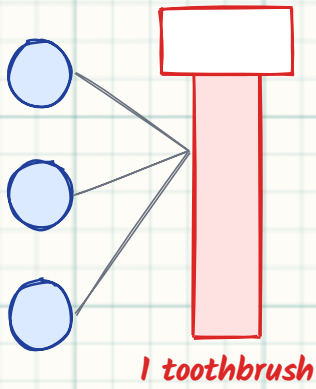
⑧ Migrating a shared database

1. Map every table → **one owner** (who writes it?)
2. Replace other services' direct reads with the owner's **API / events**
3. Move tables into the owner's **schema**, then its own **database**
4. One service at a time (Strangler Fig, Lesson 4)

⑨ ShopNest data map

Data	Owner	Others get it via
orders, order_lines	Order service	OrderPlaced events · GET /orders
payments	Payment service	PaymentCompleted event
stock levels	Inventory service	GET /stock/{id}
customer profile	Customer service	API · local copy of name in Order

⑩ Analogy



shared DB = shared toothbrush

everyone depends on it, nobody owns it

11 Interview Corner

Q. Why use a database per service?

A. Loose coupling, independent deployment and scaling, and fault isolation. A shared DB couples schemas and releases.

Q. How do you query data that lives in several services?

A. API composition, a read model built from events (CQRS), or local replicated copies. Never cross-service JOINs.

Q. What is polyglot persistence?

A. Each service uses the storage technology that best fits its data (SQL, document, search, key-value).

Q. Can two services share one database server?

A. Yes, if each has private tables or its own schema and no one touches the other's data.

Q. How do you keep data consistent across services?

A. Events and the Saga pattern with eventual consistency (Lesson 11).

⚡ Revise in 60 seconds

- → Each service **owns** its data
- → Access only via **API / events**
- → **Polyglot** persistence
- → Private tables → schema → own DB
- → No JOINs → **composition / read model**
- → **Local copies** kept fresh by events
- → No shared transaction → **Saga**
- → Migrate table by table

Next lesson → MS-11
Distributed transactions & the **Saga pattern**.
Watch on YouTube: **@TheCodeNotebook** · Notes: **thecodenotebook.com**